강화학습 Day02 hw1 보고서

작성자: Doyeon Park

작성일: 2026.10.30

목차

1. 실험 환경 및 설정
2. 학습방법
3. 학습 결과
4. Codex·GPT 사용 내역

1. 실험 환경 및 설정

항목	설정
Python	3.12.3
Gymnasium	1.4.0
NumPy	2.5.4
Matplotlib	3.11.2
환경	FrozenLake-v1, 기본 4×4 맵
미끄러짐	is_slippery=False
Q-table	16×4 배열, 초기값 0
학습 횟수	5,000 episodes
학습률 α	0.1
할인율 γ	0.95
탐험 확률 ε	max(0.05, 0.999 ** episode)
학습 행동 선택용 seed	0
평가 횟수	200 episodes
평가 방식	ε=0, Q 값 갱신 없음
평가 행동 선택용 seed	1000

FrozenLake 를 다음과 같이 MDP 로 정의하였다.

- **상태 S:** 현재 위치한 칸의 번호이며, 총 16 개이다.
- **행동 A:** 왼쪽 0, 아래 1, 오른쪽 2, 위 3 으로 총 4 개이다.
- **전이 P:** 미끄러짐을 껐으므로 선택한 방향으로 이동한다. 맵 경계를 향해 이동하면 현재 위치에 머문다.
- **보상 R:** 목표에 도착하면 1, 일반 이동과 구멍 진입은 0 이다.
- **할인율 γ:** 0.95 를 사용하여 미래 보상을 할인한다.
-

이 환경은 현재 위치와 행동을 알면 다음 상태를 결정할 수 있으므로, 상태를 현재 칸 번호로 표현할 수 있다.

Return 은 현재부터 받게 될 보상을 할인하여 합한 값이다.

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} \cdots$$

예를 들어 세 번의 행동에서 보상을 순서대로 0, 0, 1 받았다면, γ=0.95 일 때 Return 은 다음과 같다.

$$G_t = 0 + 0.95 \times 0 + 0.95^2 \times 1 = 0.9025$$

Q 값은 특정 상태에서 특정 행동을 선택했을 때 얻을 것으로 기대하는 Return 의 추정치이다. 따라서 즉시 보상이 0 인 행동도 미래의 목표 도달에 기여하면 양의 Q 값을 가질 수 있다.

한편 코드의 returns 에는 할인율을 적용하지 않은 에피소드별 보상 합계를 저장하였다. 이렇게 하면 목표에 도착한 판이 1, 나머지가 0 이므로 성공 여부를 나타내는 기록으로도 사용할 수 있다.

2. 학습방법

행동 선택에는 ε-greedy 를 사용하였다. 확률 ε 로 전체 행동 중 하나를 무작위로 선택하고, 나머지 확률$(1-\epsilon)$로 현재 Q 값이 가장 큰 행동을 선택하였다. 최대 Q 값을 가진 행동이 여러 개이면 그중 하나를 무작위로 선택하였다.
탐험 확률은 처음에 1 이며, 학습이 진행됨에 따라 감소한다. 최소값을 0.05 로 설정하여 학습 후반에도 탐험을 계속하도록 한다.

행동 후 얻은 상태·행동·보상·다음 상태를 이용하여 다음 식으로 Q 값을 갱신하였다.

$$target = r + \gamma \, max_{a'} Q(s', a')$$

$$Q(s,a) \leftarrow Q(s,a) + \alpha [target - Q(s,a)]$$

예를 들어 현재 Q 값이 0.2, 즉시 보상이 0, 다음 상태의 최대 Q 값이 0.8 이라면 다음과 같이 계산된다.

$$target = 0 + 0.95 \times 0.8 = 0.76$$

$$Q_{new} = 0.2 + 0.1(0.76 - 0.2) = 0.256$$

목표 도달이나 구멍 진입으로 실제 종료된 경우에는 이후 보상이 없으므로 목표값을 r 로 설정한다. 외부 시간 제한으로 중단된 경우에는 다음 상태의 가치를 포함하였다. 반복문은 종료 또는 중단 중 하나가 발생하면 끝내도록 구현하였다.

3. 학습 결과

3.1 학습 그래프

학습 그래프에는 에피소드별 보상과 최근 100 판의 이동평균 성공률을 표시하였고, 학습이 진행되면서 목표에 도달하는 비율이 높아지는 경향을 볼 수 있었다.

학습 후반에도 실패가 남는 이유 중 하나는 ε 의 최소값을 0.05 로 유지하여 무작위 행동을 계속 수행하기 때문이라고 유추해볼 수 있다.

(결과 사진 1 은 traning_results.png 형식으로 따로 첨부)

3.2 Q-table 및 정책 분석

Q-table heatmap 은 16 개 상태와 4 개 행동의 Q 값을 숫자와 색으로 나타낸 것이다. 목표 직전인 14 번 상태에서 오른쪽으로 이동하는 행동은 즉시 보상 1 을 받으므로 Q 값이 약 1 로 학습되었다.

구멍과 목표 상태에서는 에피소드가 종료되므로 해당 상태에서 다음 행동을 선택하지 않는다. 따라서 이 상태들의 Q 값은 모두 0 이다.

(결과 사진 2 은 q_table_heatmap.png 형식으로 따로 첨부)

정책 격자는 각 일반 상태에서 Q 값이 가장 큰 행동을 화살표로 표시하였다. 시작 상태 S, 구멍 H, 목표 G 이고, 최대 Q 값이 같은 행동이 여러 개이면 모두 표시하도록 구현하였다. 일반 칸의 Q 값이 모두 0 인 경우에는 방향을 임의로 표시하지 않고 별도로 표시하도록 하였다.

(결과 사진 3 은 policy_grid.png 형식으로 따로 첨부)

1 번 상태에서는 오른쪽보다 왼쪽의 Q 값이 높게 나타났다. 왼쪽 Q 값은 약 0.735, 오른쪽은 약 0.453 이었다. 그러나 오른쪽으로 이동하면 목표까지 5 회, 왼쪽으로 이동하면 최단 7 회의 행동이 필요하므로, 최적으로 학습된 경우에는 오른쪽의 가치가 더 높아야 한다.

이는 현재 Q-table 이 모든 상태·행동에서 최적값에 도달한 것은 아님을 보여 준다. 이러한 이유로는 상태별 경험 부족또는 불완전한 가치 전파로 설명해볼 수 있다.

3.3 학습 후 평가

학습한 Q-table 을 q_table.npy 로 저장한 뒤 다시 불러와 평가하였다. 평가에서는 ε=0 으로 설정하고 Q 값을 수정하지 않았다. 별도 환경에서 총 200 회를 실행한 결과는 다음과 같다.

평가 항목	결과
평가 횟수	200 회
성공 횟수	200 회
성공률	100%
성공 경로 길이 평균	6 회
성공 경로 길이 최소	6 회
성공 경로 길이 최대	6 회

시작 위치에서 목표까지는 아래 방향과 오른쪽 방향으로 각각 세 칸의 이동이 필요하므로, 최소 6 회의 행동이 필요하다. 평가에서 모든 성공 경로의 길이가 6 회였으므로 시작 상태에서 최단 경로로 목표에 도착했음을 알 수 있다.

4. Codex·GPT 사용 내역

강의 자료의 예시 코드를 기반으로 실습을 진행하였다. Codex·GPT 는 다음 작업에 활용하였다.

- MDP, Return, Q 값, ε-greedy 및 학습·평가 코드 해석 및 설명
- Q-table heatmap, 정책 격자 구현
- 평가 성공 경로 길이 기록과 결과 저장 기능 추가